

TASK 1:**SLAVE:**

```

1 module slave (
2     input wire      pclk,      // Clock signal
3     input wire      presetn,   // Active-low reset
4     input wire      psel,     // Select signal from master
5     input wire      penable,   // Enable signal from master
6     input wire      pwrite,    // Write signal: 1 for write, 0 for read
7     input wire [7:0] paddr,    // Address from master
8     input wire [7:0] pwdata,   // Write data from master
9     output reg [7:0] prdata,   // Read data to master
10    output reg       pready,    // Ready signal to master
11    output reg       pslverr    // Error signal to master
12 );
13
14 // Internal memory: 256 bytes
15 reg [7:0] memory[255:0];
16
17 // Address register
18 reg [7:0] addr_reg;
19
20 // Reset and initialization logic
21 always @(posedge pclk or negedge presetn) begin
22     if (!presetn) begin
23         pready    <= 1'b0;
24         pslverr   <= 1'b0;
25         addr_reg  <= 8'b0;
26     end else begin
27         // Pre-setting default values for each cycle
28         pready    <= 1'b0;
29         pslverr   <= 1'b0;
30
31         if (psel) begin
32             // Handle Read Operations
33             if (!pwrite && penable) begin
34                 if (paddr <= 8'd255) begin
35                     addr_reg <= paddr;
36                     prdata   <= memory[paddr]; // Read data from memory
37                     pready   <= 1'b1; // Indicate successful transfer
38                 end else begin
39                     pslverr <= 1'b1; // Address out of range
40                     pready <= 1'b1;
41                 end
42             end
43         end
44
45         // Handle Write Operations
46         if (pwrite && penable) begin
47             if (paddr < 8'd255) begin
48                 memory[paddr] <= pwdata; // Write data to memory
49                 pready        <= 1'b1; // Indicate successful transfer
50             end else begin
51                 pslverr <= 1'b1; // Address out of range
52                 pready <= 1'b1;
53             end
54         end
55     end
56 end
57
58 endmodule

```

MASTER:

```

1  module master (
2      input wire      presetn,          // Active-low reset
3      input wire      pclk,             // Clock signal
4      input wire      transfer,         // Transfer signal to initiate a transaction
5      input wire      read,            // Read enable signal
6      input wire      write,           // Write enable signal
7      input wire [8:0] apb_write_paddr, // Write address
8      input wire [7:0] apb_write_data,  // Write data
9      input wire [8:0] apb_read_paddr,  // Read address
10     input wire      pready,           // Slave ready signal
11     input wire      pslverr,          // Slave error signal
12     input wire [7:0] prdata,          // Data from slave during read
13
14     output reg       psel1,           // Select signal for slave 1
15     output reg       psel2,           // Select signal for slave 2
16     output reg       penable,         // Enable signal for the current transfer
17     output reg       pwrite,          // Write signal (1 = write, 0 = read)
18     output reg [8:0] paddr,           // Address signal for slave
19     output reg [7:0] pwdata,          // Data to slave during write
20     output reg [7:0] apb_read_data_out // Data output during read
21 );
22
23 // Internal state encoding
24 parameter IDLE = 2'b00;
25 parameter SETUP = 2'b01;
26 parameter ENABLE = 2'b10;
27
28 reg [1:0] state; // Current state
29 reg [1:0] next_state; // Next state
30
31 // Registered signals to hold values during ENABLE state
32 reg      psel1_reg;
33 reg      psel2_reg;
34 reg      pwrite_reg;
35 reg [8:0] paddr_reg;
36 reg [7:0] pwdata_reg;
37 reg      read_reg; // To remember if current operation is read
38 reg      transfer_reg; // Register transfer signal to detect completion
39
40 // Sequential state transition logic
41 always @(posedge pclk or negedge presetn) begin
42     if (!presetn) begin
43         state <= IDLE;
44         // Reset registered signals
45         psel1_reg <= 0;
46         psel2_reg <= 0;
47
48         pwrite_reg <= 0;
49         paddr_reg <= 9'b0;
50         pwdata_reg <= 8'b0;
51         read_reg <= 0;
52         transfer_reg <= 0;
53         apb_read_data_out <= 8'b0;
54     end else begin
55         state <= next_state;
56         transfer_reg <= transfer;
57
58         // Latch values during SETUP state
59         if (state == SETUP) begin
60             if (read && !write) begin
61                 paddr_reg <= apb_read_paddr;
62                 psel1_reg <= (apb_read_paddr[8] == 0);
63                 psel2_reg <= (apb_read_paddr[8] == 1);
64                 pwrite_reg <= 0;
65                 read_reg <= 1;
66                 pwdata_reg <= 8'b0; // Clear write data during read
67             end else if (write && !read) begin
68                 paddr_reg <= apb_write_paddr;
69                 psel1_reg <= (apb_write_paddr[8] == 0);
70                 psel2_reg <= (apb_write_paddr[8] == 1);
71                 pwrite_reg <= 1;
72                 pwdata_reg <= apb_write_data;
73                 read_reg <= 0;
74             end else begin
75                 // Neither read nor write - clear all
76                 psel1_reg <= 0;
77                 psel2_reg <= 0;
78                 pwrite_reg <= 0;
79                 paddr_reg <= 9'b0;
80                 pwdata_reg <= 8'b0;
81                 read_reg <= 0;
82             end
83         end
84
85         // Clear registered signals when transfer completes
86         if ((state == ENABLE) && pready) begin
87             // Transfer complete - clear select signals
88             // This ensures PSEL is only active during the transfer
89             if (!transfer) begin
90                 psel1_reg <= 0;
91                 psel2_reg <= 0;
92             end
93         end
94     end
95 end

```

```

93
94 // Capture read data during ENABLE state when pready is high
95 if (state == ENABLE && pready && read_reg) begin
96     apb_read_data_out <= prdata;
97 end
98 end
99 end
100 // Combinational next state logic
101 always @(*) begin
102     case (state)
103     IDLE: begin
104         if (transfer) next_state = SETUP;
105         else next_state = IDLE;
106     end
107     SETUP: begin
108         next_state = ENABLE; // Always move to ENABLE from SETUP
109     end
110     ENABLE: begin
111         if (pready) next_state = (transfer) ? SETUP : IDLE;
112         else next_state = ENABLE;
113     end
114     default: next_state = IDLE;
115 endcase
116 end
117 // Output and state-dependent control logic
118 always @(*) begin
119     // Default assignments to prevent latches
120     psel1 = 0;
121     psel2 = 0;
122     penable = 0;
123     pwrite = 0;
124     paddr = 9'b0;
125     pwrdata = 8'b0;
126     case (state)
127     IDLE: begin
128         // No activity in IDLE
129     end
130     SETUP: begin
131         penable = 0;
132         if (read && !write) begin
133             paddr = apb_read_paddr;
134             psel1 = (apb_read_paddr[8] == 0);
135             psel2 = (apb_read_paddr[8] == 1);
136             pwrite = 0;
137             pwrdata = 8'b0; // Clear data during read
138         end else if (write && !read) begin
139             paddr = apb_write_paddr;
140             psel1 = (apb_write_paddr[8] == 0);
141             psel2 = (apb_write_paddr[8] == 1);
142             pwrite = 1;
143             pwrdata = apb_write_data;
144         end else begin
145             // No operation
146             psel1 = 0;
147             psel2 = 0;
148             pwrite = 0;
149             paddr = 9'b0;
150             pwrdata = 8'b0;
151         end
152     end
153     ENABLE: begin
154         penable = 1;
155         // Use registered values to maintain signals
156         psel1 = psel1_reg;
157         psel2 = psel2_reg;
158         pwrite = pwrite_reg;
159         paddr = paddr_reg;
160         pwrdata = pwrdata_reg;
161     end
162 endcase
163 end
164 endmodule
165

```

TOP FILE:

```

1  `include "master.v" // Include the master module
2  `include "slave.v"  // Include the slave module
3
4  module apb_top (
5      input wire      pclk,           // Clock signal
6      input wire      presetn,        // Active-low reset
7      input wire      transfer,       // Signal to start transfer
8      input wire      read,           // Read control signal
9      input wire      write,          // Write control signal
10     input wire [8:0] apb_write_paddr, // Write address from master
11     input wire [7:0] apb_write_data,  // Write data from master
12     input wire [8:0] apb_read_paddr,  // Read address from master
13     output wire      pslverr,         // Slave error signal
14     output wire [7:0] apb_read_data_out // Data output during read
15 );
16 // Internal wires to connect master and slaves
17 wire      penable; // Enable signal
18 wire      pwrite;  // Write control signal
19 wire [8:0] paddr;  // Address bus
20 wire [7:0] pwdata; // Write data bus
21 wire [7:0] prdata1, prdata2; // Read data from slaves
22 wire pready1, pready2, pready; // Ready signals
23 wire psel1, psel2; // Slave select signals
24 wire pslverr_1, pslverr_2;
25 // Master instance
26 master master_inst (
27     .presetn(presetn),
28     .pclk(pclk),
29     .transfer(transfer),
30     .read(read),
31     .write(write),
32     .apb_write_paddr(apb_write_paddr),
33     .apb_read_paddr(apb_read_paddr),
34     .apb_write_data(apb_write_data),
35     .pready(pready),
36     .pslverr(pslverr),
37     .prdata(prdata1), // Combine read data buses (MUX internally based on psel)
38     .psel1(psel1),
39     .psel2(psel2),
40     .penable(penable),
41     .paddr(paddr),
42     .pwrite(pwrite),
43     .pwdata(pwdata),
44     .apb_read_data_out(apb_read_data_out)
45 );
46
47 // Slave 1 instance
48 slave slave1_inst (
49     .pclk(pclk),
50     .presetn(presetn),
51     .psel(psel1),
52     .penable(penable),
53     .pwrite(pwrite),
54     .paddr(paddr[7:0]), // Addressing within slave 1 memory range
55     .pwdata(pwdata),
56     .prdata(prdata1),
57     .pready(pready1),
58     .pslverr(pslverr_1)
59 );
60
61 // Slave 2 instance
62 slave slave2_inst (
63     .pclk(pclk),
64     .presetn(presetn),
65     .psel(psel2),
66     .penable(penable),
67     .pwrite(pwrite),
68     .paddr(paddr[7:0]), // Addressing within slave 2 memory range
69     .pwdata(pwdata),
70     .prdata(prdata2),
71     .pready(pready2),
72     .pslverr(pslverr_2)
73 );
74
75 // Combine ready signals from slaves
76 assign pready = (psel1 && pready1) || (psel2 && pready2);
77 assign pslverr = (pslverr_1 || pslverr_2);
78
79 endmodule

```

TESTBENCH:

```

1  `include "apb_top.v"
2  `timescale 1ns / 1ps
3
4  module tb_apb_top;
5
6      // Clock and reset signals
7      reg pclk;
8      reg presetn;
9
10     // Master control signals
11     reg transfer, read, write;
12     reg [8:0] apb_write_paddr;
13     reg [7:0] apb_write_data;
14     reg [8:0] apb_read_paddr;
15
16     // Outputs from the top module
17     wire pslverr;
18     wire [7:0] apb_read_data_out;
19
20     // Clock generation
21     initial begin
22         pclk = 0;
23         forever #5 pclk = ~pclk; // 100 MHz clock (10 ns period)
24     end
25
26     initial begin
27         $dumpfile("tb_apb_top.vcd");
28         $dumpvars(0, tb_apb_top);
29     end
30
31     // Instantiate the top module
32     apb_top dut (
33         .pclk(pclk),
34         .presetn(presetn),
35         .transfer(transfer),
36         .read(read),
37         .write(write),
38         .apb_write_paddr(apb_write_paddr),
39         .apb_write_data(apb_write_data),
40         .apb_read_paddr(apb_read_paddr),
41         .pslverr(pslverr),
42         .apb_read_data_out(apb_read_data_out)
43     );
44
45     // Reset and Initialization
46
47     task reset_and_init;
48     begin
49         presetn = 0;
50         transfer = 0;
51         read = 0;
52         write = 0;
53         apb_write_paddr = 9'b0;
54         apb_write_data = 8'b0;
55         apb_read_paddr = 9'b0;
56         #20; // Hold reset for 20 ns
57         presetn = 1;
58         #10;
59     end
60     endtask
61
62     // Test Case 1: Basic Write Operation
63     task test_write;
64     input [8:0] address;
65     input [7:0] data;
66     begin
67         $display("Starting Basic Write Operation Test");
68         transfer = 1;
69         write = 1;
70         read = 0;
71         apb_write_paddr = address;
72         apb_write_data = data;
73         #10; // Setup phase
74         #10; // Access phase
75         transfer = 0; // Deassert transfer signal
76         write = 0;
77         $display("Write Operation Completed: Address = %h, Data = %h", address, data);
78         #10;
79     end
80     endtask
81
82     // Test Case 2: Basic Read Operation
83     task test_read;
84     input [8:0] address;
85     begin
86         $display("Starting Basic Read Operation Test");
87         transfer = 1;
88         write = 0;
89         read = 1;
90         apb_read_paddr = address;
91         #10; // Setup phase

```

```

91         #10; // Access phase
92         $display("Read Operation Completed: Address = %h, Data Read = %h", address,
93             apb_read_data_out);
94         transfer = 0; // Deassert transfer signal
95         read = 0;
96         #10;
97     end
98 endtask
99
100 // Test Case 3: Address Decoding (Slave Selection)
101 task test_address_decoding;
102 begin
103     $display("Starting Address Decoding Test");
104     test_write(9'h005, 8'hA5); // Target Slave 2
105     test_write(9'h085, 8'h5A); // Target Slave 1
106 end
107 endtask
108
109 // Test Case 4: Write with Wait States
110 task test_write_with_wait_states;
111 input [8:0] address;
112 input [7:0] data;
113 begin
114     $display("Starting Write Operation with Wait States Test");
115     transfer = 1;
116     write = 1;
117     read = 0;
118     apb_write_paddr = address;
119     apb_write_data = data;
120     #10; // Setup phase
121     // Simulate a wait state by delaying slave response
122     @(posedge pclk);
123     $display("Slave ready signal delayed");
124     #20; // Access phase
125     transfer = 0;
126     write = 0;
127     $display("Write Completed with Wait States: Address = %h, Data = %h", address, data);
128     #10;
129 end
130 endtask
131
132 // Test Case 5: Read with Wait States
133 task test_read_with_wait_states;
134 input [8:0] address;
135 begin
136     $display("Starting Read Operation with Wait States Test");
137     transfer = 1;
138     write = 0;
139     read = 1;
140     apb_read_paddr = address;
141     #10; // Setup phase
142     // Simulate a wait state by delaying slave response
143     @(posedge pclk);
144     $display("Slave ready signal delayed");
145     #20; // Access phase
146     transfer = 0;
147     read = 0;
148     $display("Read Completed with Wait States: Address = %h, Data Read = %h", address,
149         apb_read_data_out);
150     #10;
151 end
152 endtask
153
154 // Test Case 6: Error Handling (PSLVERR)
155 task test_error_handling;
156 input [8:0] invalid_address;
157 begin
158     $display("Starting Error Handling Test");
159     transfer = 1;
160     write = 1;
161     read = 0;
162     apb_write_paddr = invalid_address;
163     apb_write_data = 8'hFF;
164     #10; // Setup phase
165     #10; // Access phase
166     transfer = 0;
167     write = 0;
168     $display("Error Condition PSLVERR = %b for Invalid Address = %h", pslverr, invalid_address);
169     #10;
170 end
171 endtask
172
173 // Test Case 7: Burst Transfers
174 task test_burst_transfers;
175 begin
176     $display("Starting Burst Transfers Test");
177     test_write(9'h001, 8'h11);
178     test_read(9'h001);
179     test_write(9'h002, 8'h22);
180     test_read(9'h002);

```

```

181     test_write(9'h003, 8'h33);
182     test_read(9'h003);
183     $display("Burst Transfers Completed");
184     end
185 endtask
186
187 // Case 8: Test Out-of-Range Address Handling
188 task test_out_of_range_address;
189     input [8:0] invalid_address;
190     begin
191         $display("\n[Case 8] Testing Out-of-Range Address Handling...");
192         transfer = 1;
193         write = 1;
194         read = 0;
195         apb_write_paddr = invalid_address; // Use an invalid address
196         apb_write_data = 8'hFF; // Some test data
197         #10; // Setup phase
198         #10; // Access phase
199         transfer = 0;
200         write = 0;
201         if (pslverr)
202             $display("Error detected for invalid address %h, PSLVERR = %b", invalid_address, pslverr);
203         else
204             $display(
205                 "No error detected for invalid address %h, PSLVERR = %b", invalid_address, pslverr
206             );
207         #10;
208     end
209 endtask
210
211 // Case 9: Test Reset Behavior
212 task test_reset_behavior;
213     begin
214         $display("\n[Case 9] Testing Reset Behavior...");
215         presetn = 0; // Apply reset
216         #20;
217         if (!presetn) $display("System Reset Asserted.");
218         presetn = 1; // Release reset
219         #10;
220         if (presetn)
221             $display("System Reset Released. Registers and signals should return to default states.");
222         #10;
223     end
224 endtask
225
226 // Case 10: Test Randomized Transactions
227 task test_randomized_transactions;
228     integer i;
229     begin
230         $display("\n[Case 10] Stress Testing with Randomized Transactions...");
231         for (i = 0; i < 20; i = i + 1) begin
232             transfer = 1;
233             // Randomly pick between read and write
234             if ($random % 2) begin
235                 write = 1;
236                 read = 0;
237                 apb_write_paddr = $random % 9'h100; // Random 9-bit address
238                 apb_write_data = $random % 8'hFF; // Random 8-bit data
239                 $display("Write Transaction: Address = %h, Data = %h", apb_write_paddr, apb_write_data);
240             end else begin
241                 write = 0;
242                 read = 1;
243                 apb_read_paddr = $random % 9'h100; // Random 9-bit address
244                 $display("Read Transaction: Address = %h", apb_read_paddr);
245             end
246             #10; // Setup phase
247             #10; // Access phase
248             transfer = 0;
249             write = 0;
250             read = 0;
251             #10;
252         end
253         $display("Randomized Transactions Completed.");
254     end
255 endtask
256
257 // Main Testbench Execution
258 initial begin
259     $display("APB System Testbench Start");
260     reset_and_init;
261
262     test_write(9'h005, 8'hAA); // Test Case 1
263     test_read(9'h005); // Test Case 2
264     test_address_decoding; // Test Case 3
265     test_write_with_wait_states(9'h010, 8'hBB); // Test Case 4
266     test_read_with_wait_states(9'h010); // Test Case 5
267     test_error_handling(9'h1FF); // Test Case 6
268     test_burst_transfers; // Test Case 7
269     test_out_of_range_address(9'h1FF); // Test Case 8
270     test_reset_behavior; // Test Case 9
271     test_randomized_transactions; // Test Case 10
272     $display("APB System Testbench Complete");
273     $finish;
274 end
275
276 endmodule

```

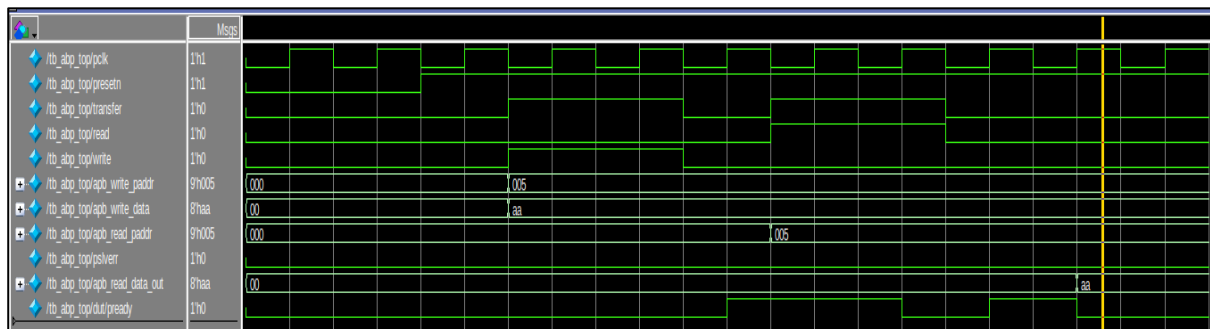
TC1 — Basic Write Operation

This test verifies that the APB master can perform a normal write transaction. A valid address and data are provided, and the selected slave should receive and store the data correctly without any error.



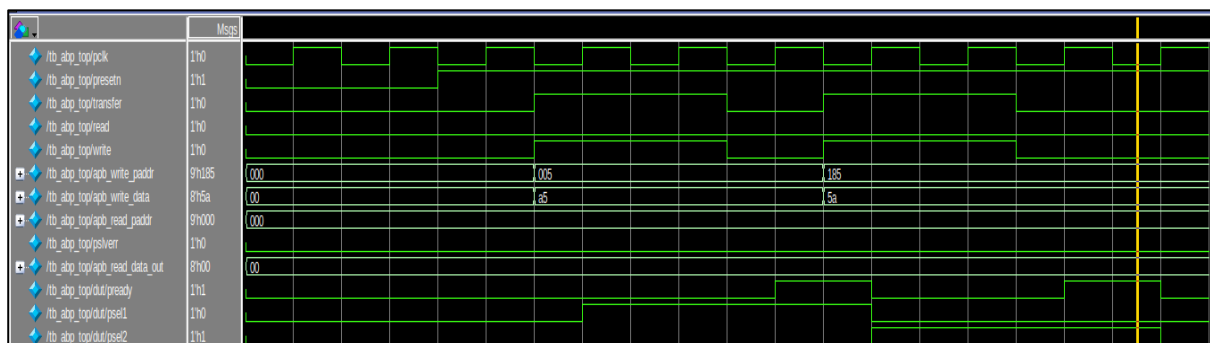
TC2 — Basic Read Operation

This test verifies a normal APB read transaction. The master requests data from a valid address, and the slave should return the stored data through “apb_read_data_out”.



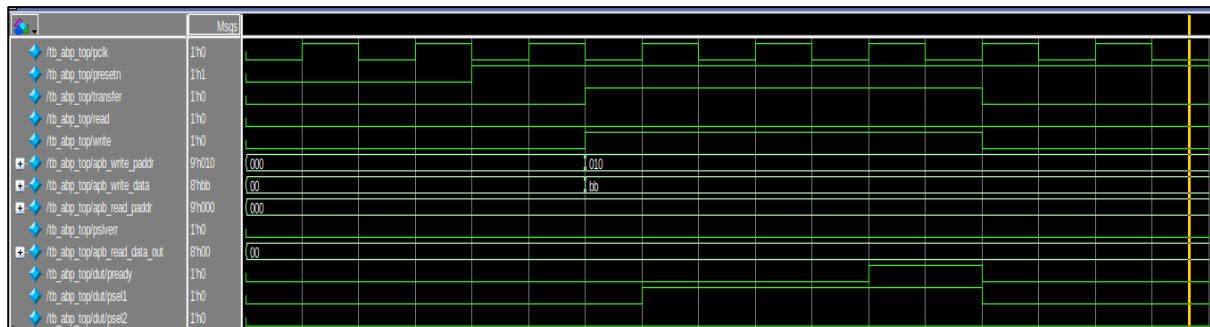
TC3 — Address Decoding

This test checks whether the address decoder selects the correct slave based on the given address. Only the slave corresponding to the address range should be activated, while the other slave remains inactive.

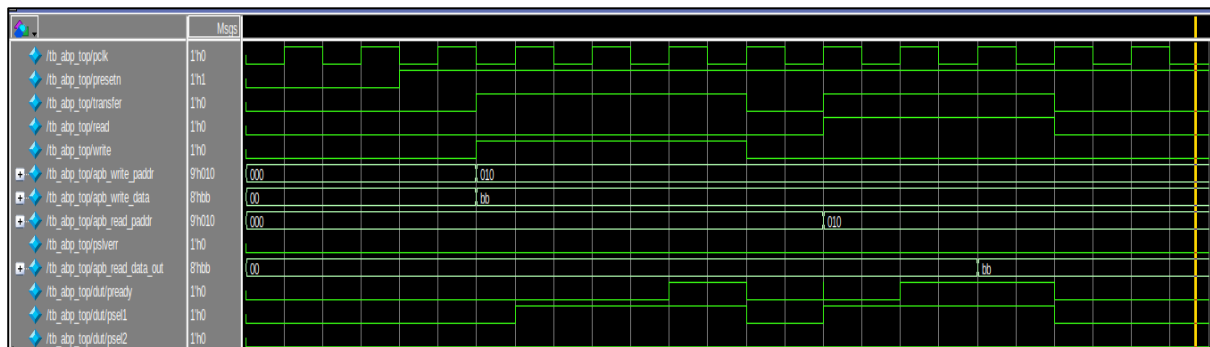


TC4 — Write with Wait States

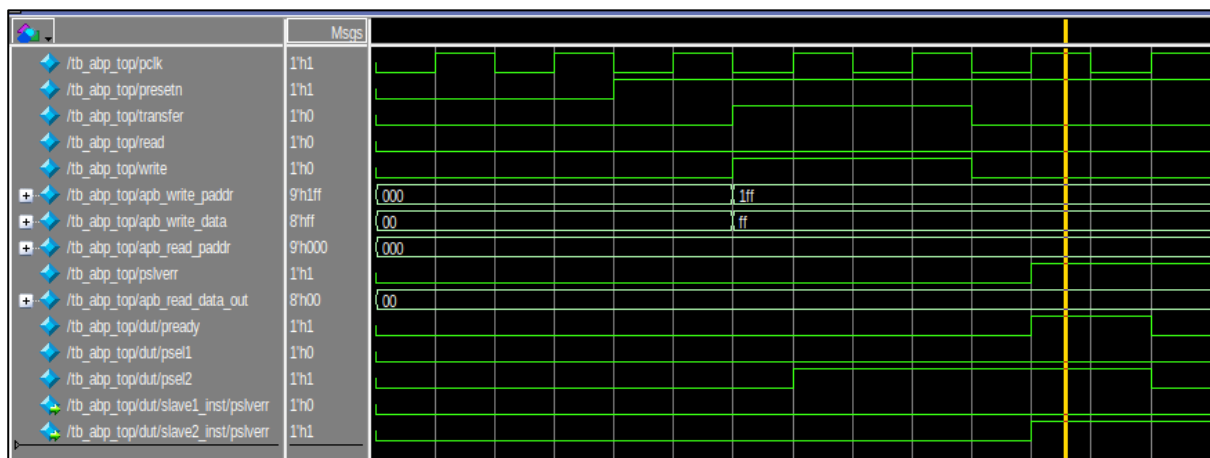
This test verifies the behaviour of a write transaction when the slave introduces a delay. The APB master should remain in the transfer state until the slave indicates that it is ready, after which the write operation completes.

**TC5 — Read with Wait States**

This test checks a read transaction where the slave delays its response. The master should wait until the slave becomes ready and then capture the correct read data.

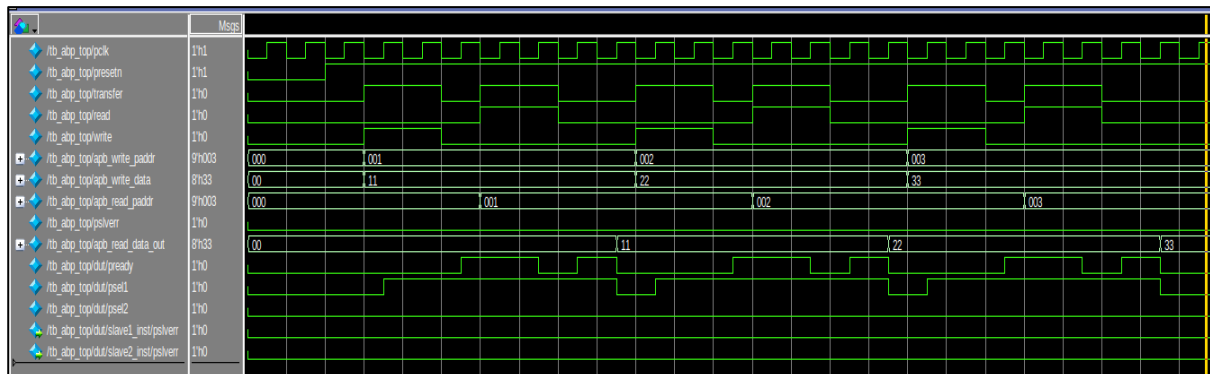
**TC6 — Error Handling**

This test verifies how the system handles an invalid address or error condition. The slave should assert “pslverr”, allowing the master or top module to detect that the transaction was unsuccessful.



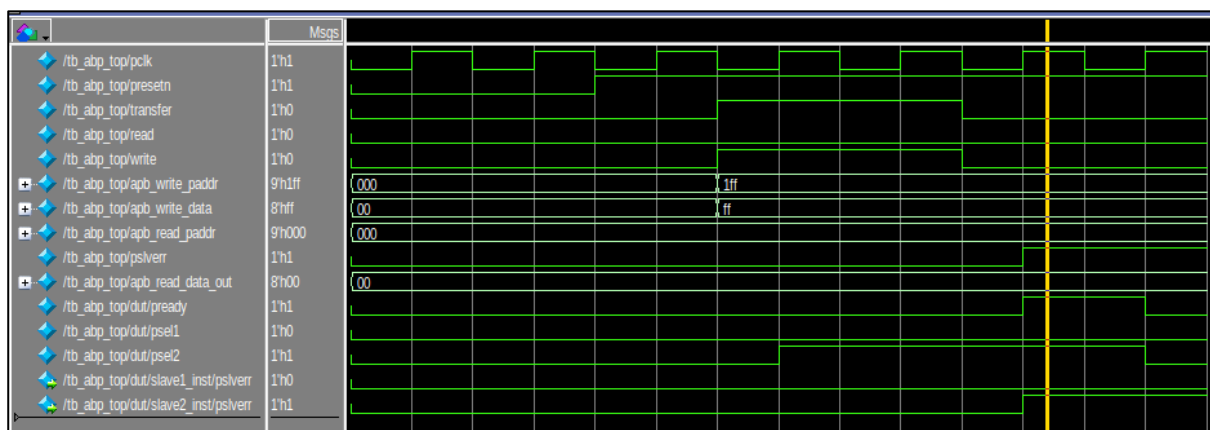
TC7 — Burst Transfers

This test performs multiple consecutive read and write operations. It verifies that the APB system can handle back-to-back transactions correctly without losing data or entering an incorrect state.



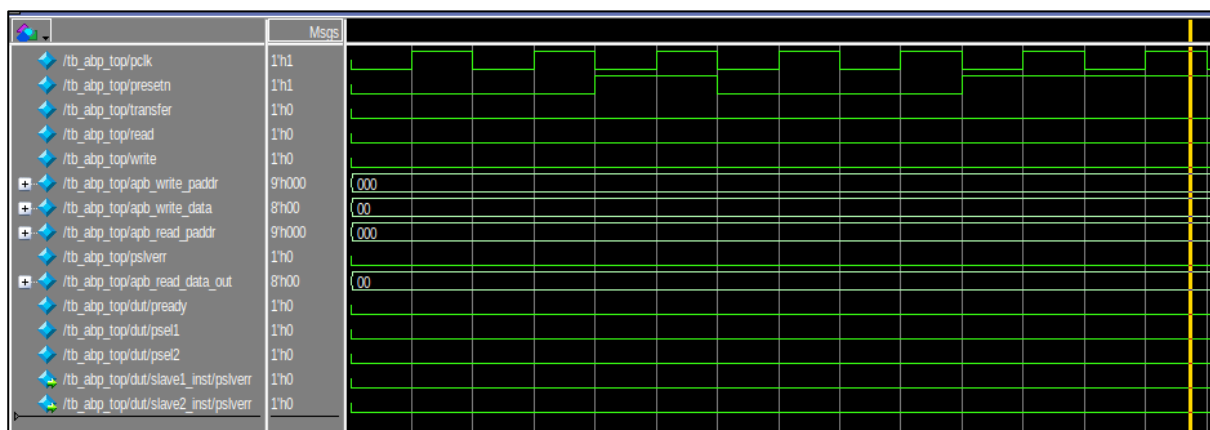
TC8 — Out-of-Range Address

This test sends a transaction to an address outside the valid address range. The expected behaviour is that the system detects the invalid address and asserts “pslverr”.



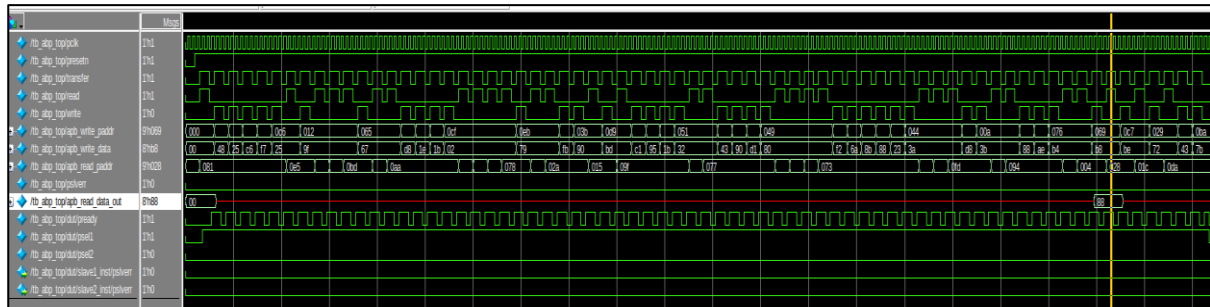
TC9 — Reset Behaviour

This test verifies the active-low reset functionality. When “present” is asserted low, the system should return to its default state, clear or reset its control signals, and prevent any active transaction.



TC10 — Randomized Transactions

This test performs multiple randomly generated read and write transactions with different addresses and data values. It is used to check the overall stability and reliability of the APB system under varying transaction conditions.



TASK 2:

Slave:

```

2  module counter_slave (
3      input wire clk,
4      input wire rst,
5      input wire out_ready,
6      output reg out_valid,
7      output reg [7:0] out_data = 8'd0
8  );
9
10 always @(posedge clk) begin
11     if (rst) begin
12         out_valid <= 1'b0;
13         out_data <= 8'd0;
14     end else begin
15         out_valid <= 1'b1;
16         if (out_ready) out_data <= out_data + 8'd1;
17     end
18 end
19 endmodule
20

```

Testbench:

```

2  module tb_handshake;
3      reg clk_tb;
4      reg rst_tb;
5      reg out_ready_tb;
6      wire out_valid_tb;
7      wire [7:0] out_data_tb;
8
9      counter_slave uut
10     (
11         .clk(clk_tb),
12         .rst(rst_tb),
13         .out_ready(out_ready_tb),
14         .out_valid(out_valid_tb),
15         .out_data(out_data_tb)
16     );
17
18     always #5 clk_tb = ~clk_tb;
19

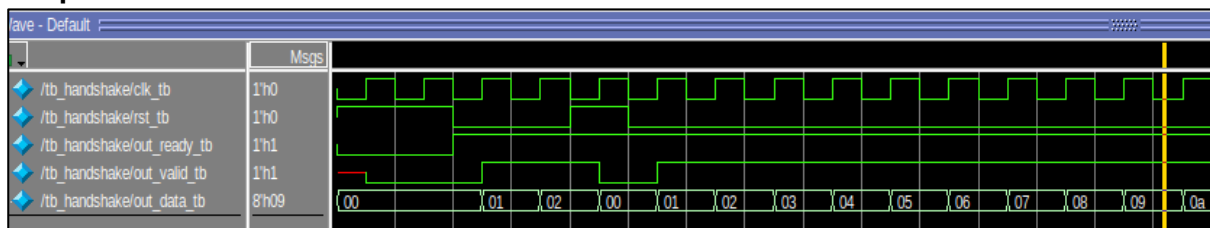
```

```

20      initial begin
21          $monitor("Time=0%t reset=%b valid=%b ready=%b data=%b",
22                  $time, rst_tb, out_valid_tb, out_ready_tb, out_data_tb);
23
24          clk_tb = 1'b0;
25          out_ready_tb = 1'b0;
26          rst_tb = 1'b1;
27          #20;
28
29          rst_tb = 1'b0;
30          out_ready_tb = 1'b1;
31          #10;
32
33          rst_tb = 1'b0;
34          out_ready_tb = 1'b1;
35          #10;
36
37          rst_tb = 1'b1;
38          out_ready_tb = 1'b1;
39          #10;
40
41          rst_tb = 1'b0;
42          out_ready_tb = 1'b1;
43          #100
44
45          $finish;
46      end
47  endmodule

```

Output Wave:



F4PGA Build:

```

kali@kali:~/f4pga-examples$ cd f4pga-examples
kali@kali:~/f4pga-examples$ export F4PGA_INSTALL_DIR=/opt/f4pga
kali@kali:~/f4pga-examples$ export FPGA_FAM="xc7"
kali@kali:~/f4pga-examples$ source "$F4PGA_INSTALL_DIR/$FPGA_FAM/conda/etc/profile.d/conda.sh"
kali@kali:~/f4pga-examples$ conda activate xc7
(xc7) kali@kali:~/f4pga-examples$ cd xc7
(xc7) kali@kali:~/f4pga-examples/xc7$ export F4PGA_INSTALL_DIR=/opt/f4pga
(xc7) kali@kali:~/f4pga-examples/xc7$ export FPGA_FAM="xc7"
(xc7) kali@kali:~/f4pga-examples/xc7$ TARGET="arty_35" make -C counter_slave
make: Entering directory '/home/kali/f4pga-examples/xc7/counter_slave'
make: 'build' is up to date.
make: Leaving directory '/home/kali/f4pga-examples/xc7/counter_slave'
(xc7) kali@kali:~/f4pga-examples/xc7$ ls -l /dev/ttyUSB* /dev/ttyACM* /dev/ttyS* 2>/dev/null
crw-rw---- 1 root dialout 4, 64 Aug 14 23:27 /dev/ttyS0

```

Bitstream Upload:

```

(xc7) kali@kali:~/f4pga-examples/xc7$ python3 esp32_fpga_program.py /dev/ttyUSB0 counter_slave/build/arty_35/counter_slave.bit
=====
ESP32 -> XC7A35T JTAG Programmer
=====
Reading bitstream...
Payload offset : 0x68
Payload size   : 2192012 bytes
Opening /dev/ttyUSB0 at 921600 baud...
1. Resetting JTAG TAP...
2. Sending JPROGRAM...
3. Waiting for FPGA initialization...
4. Sending CFG IN...
5. Sending FPGA bitstream...
Programming 2192012 bytes...
Programming: 100.00% Elapsed: 53.4s
Bitstream programming finished in 53.38 seconds
6. Sending JSTART...
7. Sending startup clocks...
=====
FPGA PROGRAMMING COMPLETE
=====
(xc7) kali@kali:~/f4pga-examples/xc7$

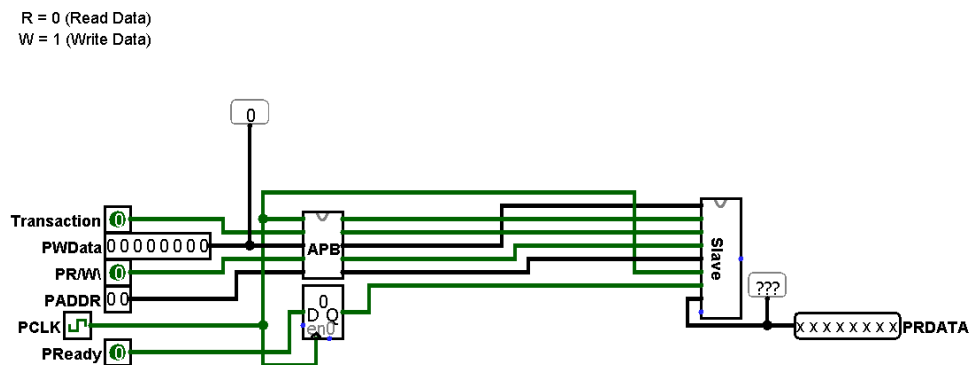
```

On the FPGA, the handshake protocol is demonstrated using the “counter_slave” module. When the active high reset (rst) is asserted, the slave clears out_data to 0 and de-asserts “out_valid”, indicating that no valid data is available. After reset is released, “out_valid” becomes HIGH, indicating that the slave is presenting valid data. The “out_data” counter increments only when “out_ready” is HIGH on the active clock edge. This follows the standard valid-ready handshake protocol: out_valid indicates that the slave has valid data available, while out_ready indicates that the receiver is ready to accept it. Therefore, data transfer occurs only when out_valid && out_ready is true. If out_ready is LOW, the counter holds its current value, demonstrating that the slave waits until the receiver is ready before sending the next data value.

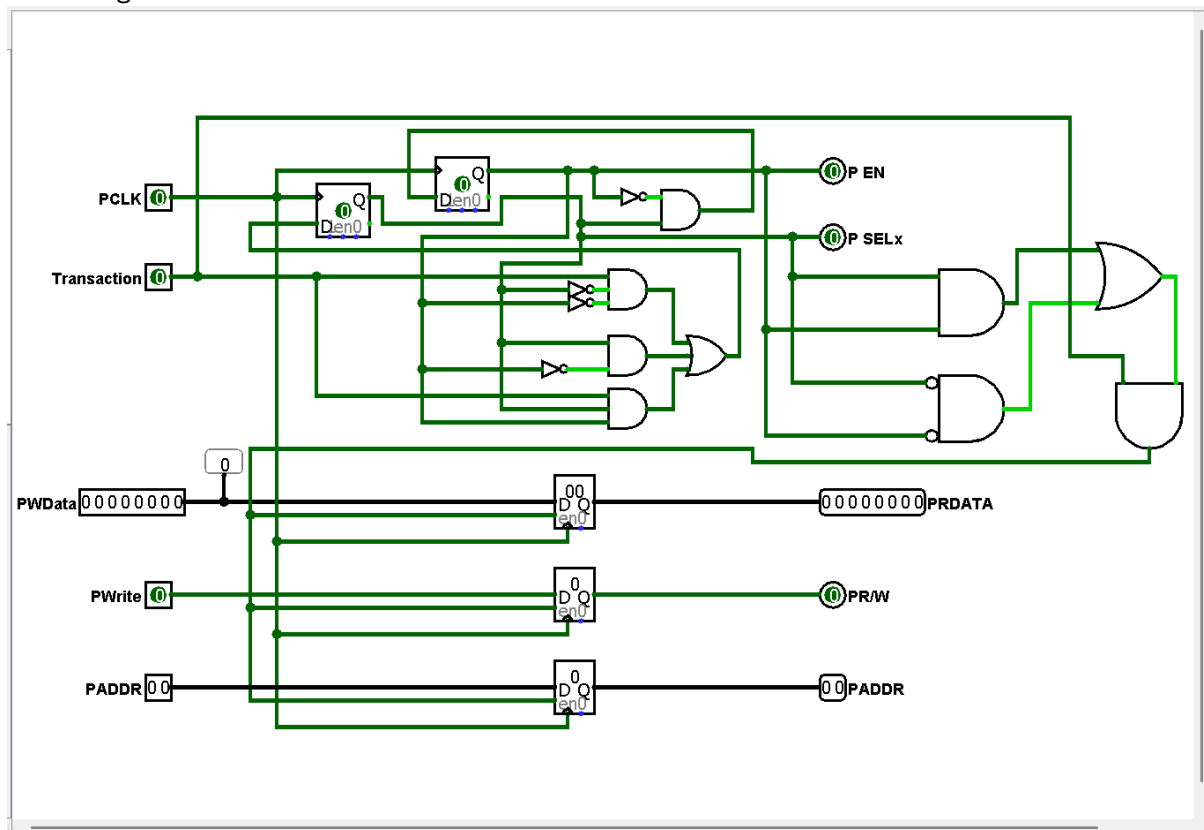
Task 3:

Single Master Single Slave:

APB Top:



APB Bridge:



APB Slave:

